GAZİ UNIVERSITY

FACULTY OF ENGINEERING
DEPARTMENT OF INDUSTRIAL ENGINEERING

Lecturer : Dr Ercan Ezin

IE104-COMPUTER PROGRAMMING I

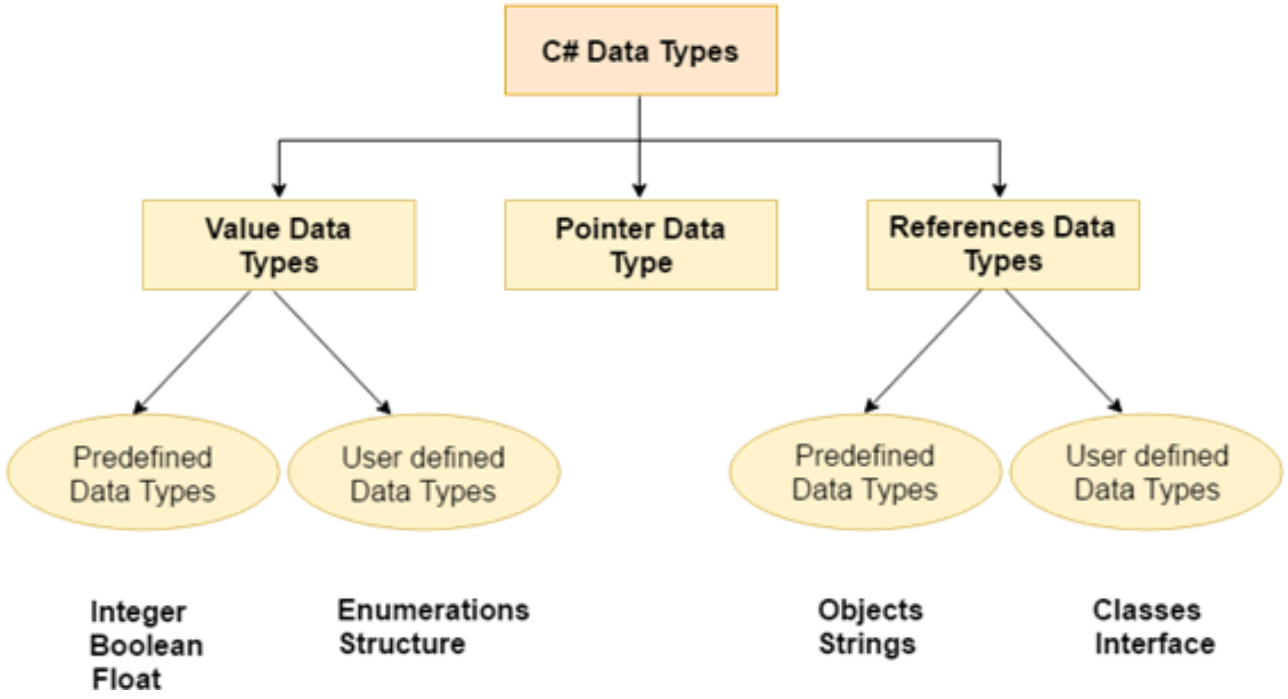
WEEK 4: VARIABLES AND DATA TYPES

Assignment

- Did you work on the examples given on course website?

Basic Data Types in C#

- There are three popular data types in C# language.

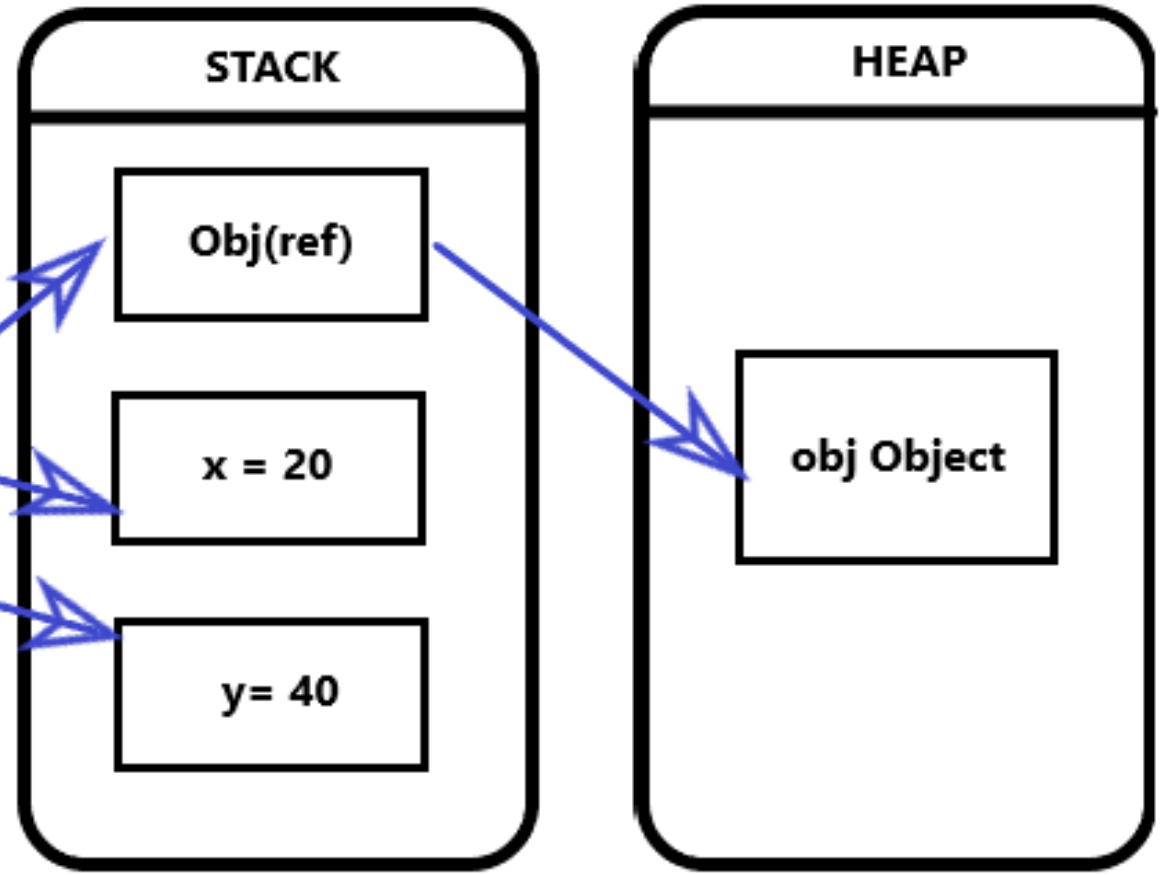


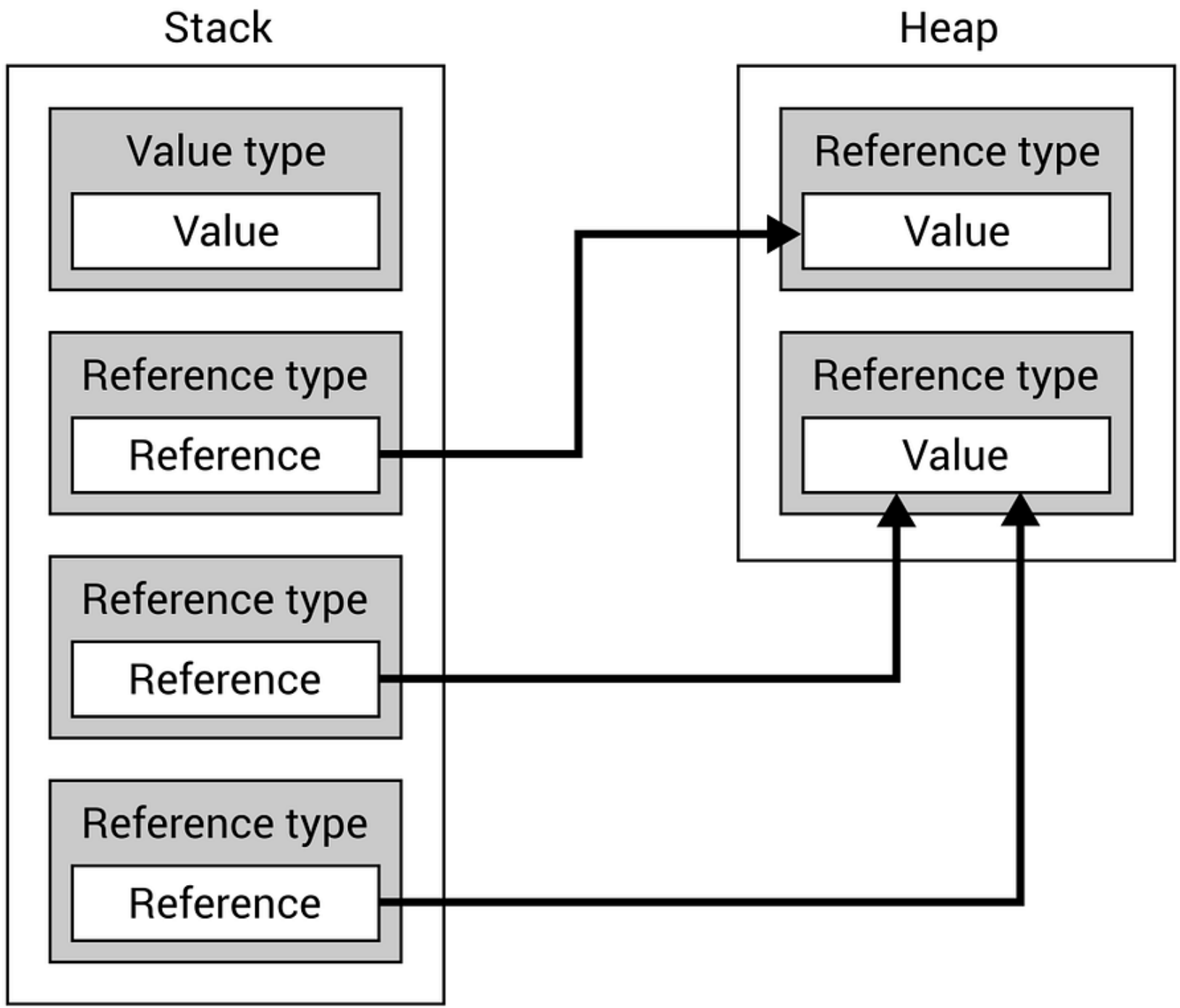
Types	Data Types
Value Data Type	short, int, char, float, double etc
Reference Data Type	String, Class, Object and Interface
Pointer Data Type	Pointers

Basic Data Types in C#

- Keeping the data in memory is done by one of 6 regions. These are:
- **Stack Region:** In the program, an integer type object is loaded at runtime.
- It is the memory area where the defined variables are kept. The allocation of space to the variables by the compiler should be known beforehand.
- **Heap Region:** All C# objects are created in this region. Unlike the Stack, the object to be allocated in this region is not required to be known by the compiler. The new keyword is used to allocate a space to an object in his region.
- Spaces allocated with **'new'** keyword are dynamic and placed in Heap. Allocated at runtime, not allocated at compile time. **It is slower than Stack.**
- Value types are kept in Stack, Reference data types are kept in Heap.

```
public void metot()
{

    int x = 20;

    int y = 40;

    Class1 obj = new Class1();

}
```




Stack
Heap
Value type
Value
Reference type
Reference
Reference type
Reference
Reference type
Reference
Reference type
Value
Reference type
Value

Basic Data Types in C#

- **Register Region:** Registers are special structures on the microprocessor. Thanks to their structures, data transfer to other regions can be done more quickly.
- **Static Region:** Represents any region in memory. Data held in static areas is stored for the entire runtime of the program. The **'static'** keyword is used to give this property to an object.
- **Fixed Region:** Area in a program where the values remain the same.
- **Non-RAM Region:** Represents disk areas that are out of memory region.

Defining a Variable

Variable definition is as follows:

<datatype> <name>

- In C#, it is restricted to use a variable without assigning any value.
- If we want to use a variable, it is a mandatory to assign a value to thhe variable.
- Definitions can be made at any section of the program. There is no rule about at this point.

Defining a Variable

Basic rules of variable naming are:

- There is **case sensitivity** in variable naming in C#.
- Variable names **cannot start** with a **numeric character such as 1,0 etc.**.
- Variable names **cannot contain spaces**.
- A defined variable **can only be accessed** from within the block in which it is defined. This block is called the variable's scope.
- A variable **defined as a class member** is always **valid** within the class's activity area.
- A local variable can be active; **as long as it stays i**n the scope in which it is defined.
- Variables defined in a loop block are in a transaction scope until they are excluded from the loop.

Scopes of Variables

```
using System;

namespace IE_Gazi
{
    0 başvuru
    class Program
    {
        0 başvuru
        static void Main(string[] args)
        {
            {
                int num1 = 5;
                Console.WriteLine(num1);
            }
            {
                int num1 = 11;
                Console.WriteLine(num1);
            }
            Console.ReadKey();
        }
    }
}
```

C:\Users\AKIN\source\repos\IE_Gazi\IE_Gazi\bin\Debug...

```
5
11
```

Scopes of Variables

- Redefining a variable before resuming the scope activity will cause an error during compilation.

The same variable redefined

```
using System;

namespace IE_Gazi
{
    0 başvuru
    class Program
    {
        0 başvuru
        static void Main(string[] args)
        {
            int num1;
            {
                int num1 = 5;
                Console.WriteLine(num1);
            }

            Console.ReadKey();
        }
    }
}
```

Program.cs | IE_Gazi | IE_Gazi.Program | Main(string[] args)

33 % | 1 | 1 | Sat: 9 | Krkt: 22

Hata Listesi

Tüm Çözüm | 1 Hata | 1 Uyarı | 0 / 1 İleti | Derleme + IntelliSense | Hata Listesinde Ara

Kod	Açıklama	Proje	Dosya	Çi...	Gizleme Du
CS0136	'num1' adlı bir yerel veya parametre, bu ad bir kapanış yerel kapsamında bir yereli veya parametreyi tanımlamak için kullanıldığından bu kapsamda ifade edilemiyor	IE_Gazi	Program.cs	11	Etkin
CS0168	'num1' değişkeni ifade edilir ancak hiçbir zaman kullanılmaz	IE_Gazi	Program.cs	9	Etkin

Constants

- The data whose value is not planned to change throughout the program are defined as a constant. To create a constant value, **'const'** keyword is used before the variable name.
- Constant values should be used when **initializing** constants.
- As with defining variables, initial values must be given when defining constants.

```
const double pi=3.14
```

Constants

Program.cs*

IE_Gazi | IE_Gazi.Program

```
1   using System;
2
3   namespace IE_Gazi
4   {
        0 başvuru
5       class Program
6       {
            0 başvuru
7           static void Main(string[] args)
8           {
9               int num1=5, num2=15;
10              const int c = num1 + num2;
11                  Console.WriteLine(c);
12
13              Console.ReadKey();
14          }
15      }
16  }
17
```

133 % | 1 | 0

Hata Listesi...

Tüm Çözüm | 1 Hata | 0 Uyarı | 0 / 1 İleti | Derleme + IntelliSense

	Kod	Açıklama
✕	CS0133	'c' atanan ifade sabit olmalıdır

Constants

Program.cs

IE_Gazi | IE_Gazi.Program

```
1   using System;
2
3   namespace IE_Gazi
4   {
        0 başvuru
5       class Program
6       {
            0 başvuru
7           static void Main(string[] args)
8           {
9
10              const int c1 = 10, c2=20;
11              const int c3 = c1 + c2;
12                  Console.WriteLine(c3);
13
14              Console.ReadKey();
15          }
16      }
17  }
18
```

C:\Users\AKIN\source\repos\IE_Gazi\IE_Gazi\bin\Debug\netcoreapp...

```
30
```

Program.cs

IE_Gazi | IE_Gazi.Program

```
1   using System;
2
3   namespace IE_Gazi
4   {
        0 başvuru
5       class Program
6       {
            0 başvuru
7           static void Main(string[] args)
8           {
9
10              const int c1 = 10, c2=20;
11              c1 = c1 + 2;
12              const int c3 = c1 + c2;
13                  Console.WriteLine(c3);
14
15              Console.ReadKey();
16          }
17      }
18  }
```

133 % | 1 | 0

Hata Listesi

Tüm Çözüm | 1 Hata | 0 Uyarı | 0 / 1 İleti | Derleme + IntelliSense

Kod	Açıklama
CS0131	Atamanın sol tarafındaki değişken, özellik veya dizin oluşturucu olmalıdır

Constants

There are 3 basic rules regarding constants. These are:

- Constants must be assigned when they are defined. Non-initialized variables cannot be constants.
- Constant expressions can only be initialized with constant expressions.
- Also, the static keyword cannot be not used, because the constant expressions are a static object due to their structure.

Value and Reference Types

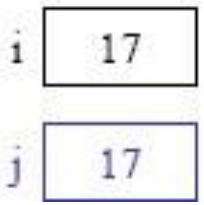
- Value types get the value of a variable directly from the memory region.
- Reference types are used to point to another object. The reference types store the addresses of the objects created in the heap space.
- Value types are created in the stack.
- Reference types are stored in the heap memory region which is a bit more limited in use.

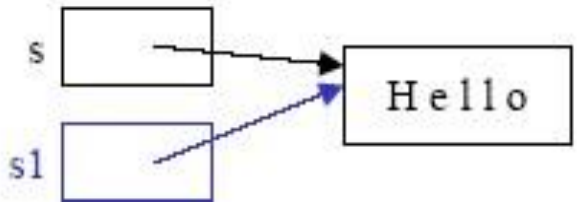
Value and Reference Types

- Basic data types (int,double,float...) are value types.
- Any class type is a reference type.
- When equalizing two value type objects, the values stored in the variables are assigned by copying, and in this case two new independent objects are obtained. Changing the value of one does not affect the other.
- However, when two reference types are equalized, the data held in these objects are not copied, the addresses of these objects in the heap region are processed.
- Since two objects point to the same location in the stack region, any change to one will affect the other.

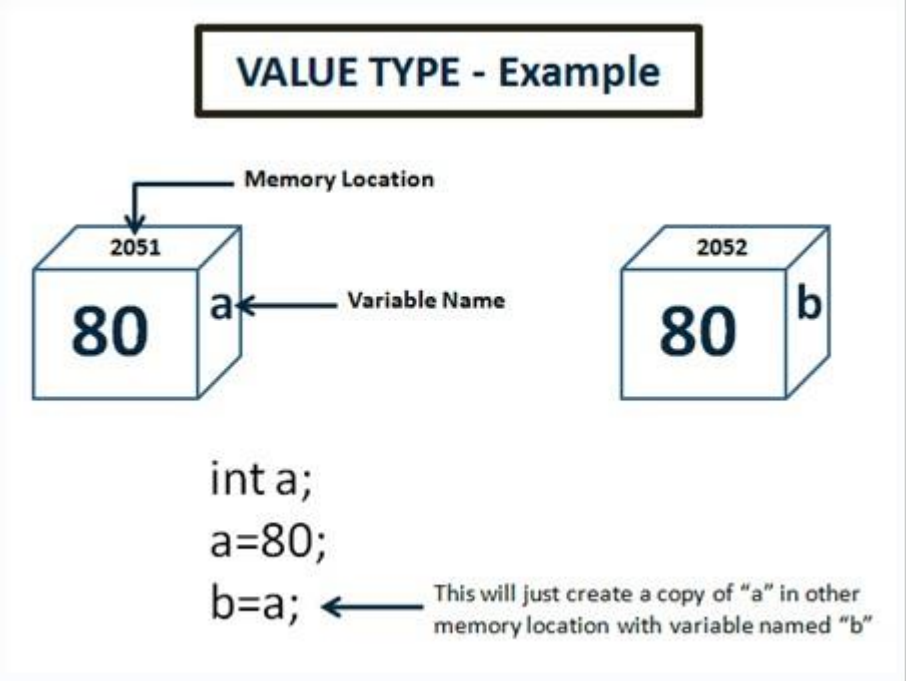
Value and Reference Types

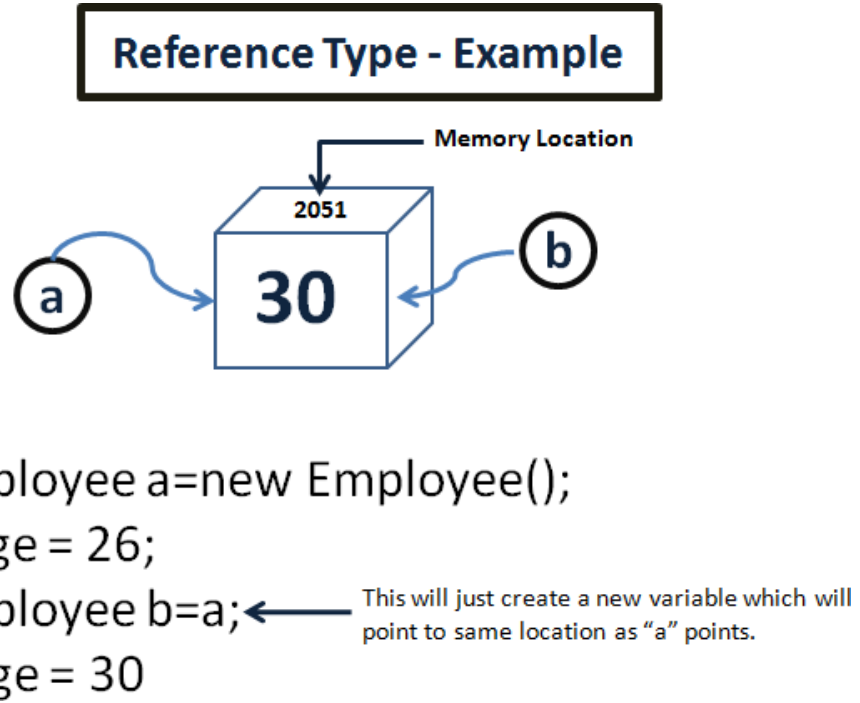
	Value Types	Reference Types
variable contains	value	reference
stored on	stack	heap
initialisation	0, false, '\0'	null
assignment	copies the value	copies the reference
example	int i = 17; int j = i;	string s = "Hello"; string s1 = s;





Value and Reference Types





Value and Reference Types

- Thanks to the CTS, all languages developed for the .NET platform use the same data types.
- There are 15 predefined core datatypes in C# (13 of them are value types, 2 of them are reference types)
- All value types derive from an object named **Object**. Both an object or a data type in C# is actually an **Object**.
- In value types, the value of an object is directly stored. Defined value types can be initialized as follows.

```
int a=3,b; b=a;
```

- At this point, the **b value** will not be affected by changes on **a value**.

Value and Reference Types

- Initializing value types;

```
a=new int();//the constructor works.(reference type)
a=0;
```

- It does the same in the two lines above.

```
float b;//compiler error, must be assigned.
```

Error: Use of unassigned local variable b

```
float b=new float(); // no error will be given since constructor worked
b=3.21f//new assignment is made
```

Value and Reference Types

Type	Represents	Range	Default Value
bool	Boolean value	True or False	False
byte	8-bit unsigned integer	0 to 255	0
char	16-bit Unicode character	U +0000 to U +ffff	'\0'
decimal	128-bit precise decimal values with 28-29 significant digits	$(-7.9 \times 10^{28}$ to $7.9 \times 10^{28}) / 10^{0 \text{ to } 28}$	0.0M
double	64-bit double-precision floating point type	$(+/-)5.0 \times 10^{-324}$ to $(+/-)1.7 \times 10^{308}$	0.0D
float	32-bit single-precision floating point type	-3.4×10^{38} to $+ 3.4 \times 10^{38}$	0.0F
int	32-bit signed integer type	-2,147,483,648 to 2,147,483,647	0
long	64-bit signed integer type	-923,372,036,854,775,808 to 9,223,372,036,854,775,807	0L
sbyte	8-bit signed integer type	-128 to 127	0
short	16-bit signed integer type	-32,768 to 32,767	0
uint	32-bit unsigned integer type	0 to 4,294,967,295	0
ulong	64-bit unsigned integer type	0 to 18,446,744,073,709,551,615	0
ushort	16-bit unsigned integer type	0 to 65,535	0

An Example

- Try this code yourself.

```
namespace IE_Gazi
{
    0 başvuru
    class Program
    {
        0 başvuru
        static void Main(string[] args)
        {

            bool a = new bool();
            byte a1 = new byte();
            char a2 = new char();
            decimal a3 = new decimal();
            double a4 = new double();
            float a5 = new float();
            Console.WriteLine(a);
            Console.WriteLine(a1);
            Console.WriteLine(a2);
            Console.WriteLine(a3);
            Console.WriteLine(a4);
            Console.WriteLine(a5);

            Console.ReadKey();
        }
    }
}
```

```
C:\Users\AKIN\source\repos\IE_Gazi\IE_Ga
False
0

0
0
0
```

Reference Types – Object

- There are two predefined reference types in C#

  - **Object**
  - **String**

- All the data types are derived from the **OBJECT** type. So, we can assign any data type to the Object type.
- **Object type** can be used for different purposes by customizing.
  - Matching to Object (Boxing)
  - Vice versa, transforming Object (Unboxing).

Reference Types – String

- Strings of reference type should be treated as a string of Unicode characters.

  - String1 = “Hello”;
  - String2 = “ .NET”;
  - String3 = String1 + String2;
  - String3 = “Hello .NET”;

- The + operator is used to add strings one after the other.

Reference Types - String

- We use the '\' expression to denote characters with the special meanings (escape character).
- String path="C:\\WINDOWS\\assembly"
- We can use @ sign to tell compiler to enclose a string as it appears in the code, for that we place @ in front of the string expression.
- For example normally we would have to use double escape characters but since we placed @ sign, complier will use string as is
- Example following two strings are same for compiler:
    - String path=@"C:\WINDOWS\assembly"
    - String path="C:\\WINDOWS\\assembly"

Data Types

- Try this code yourself.

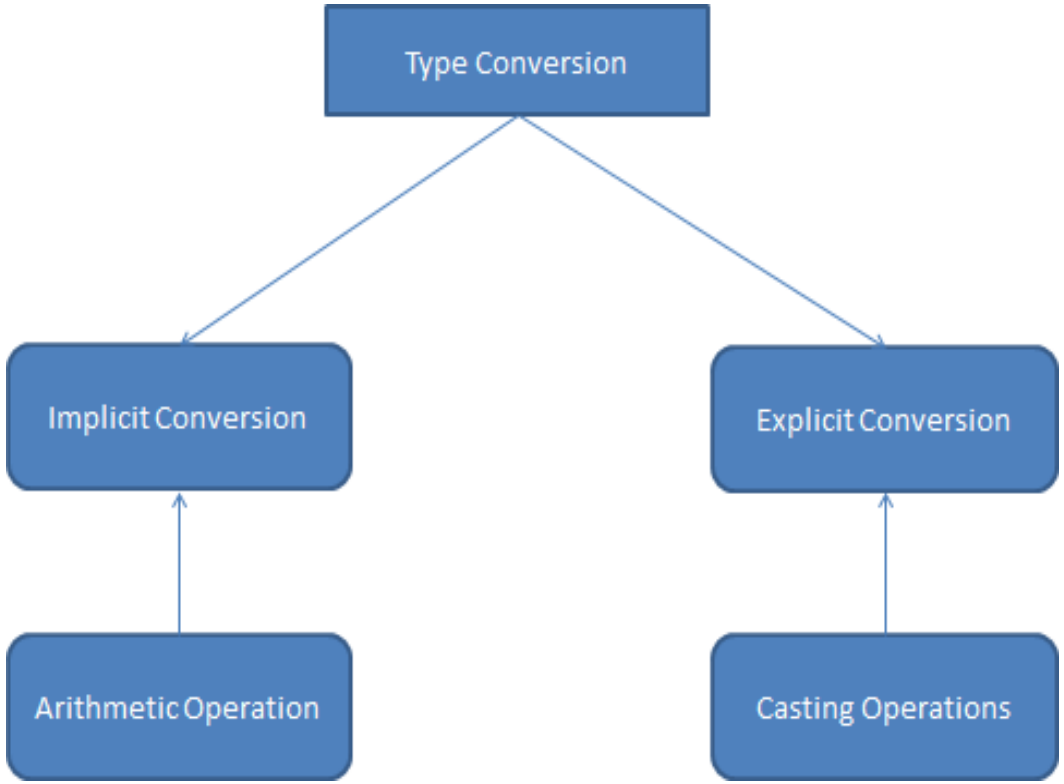
```
namespace IE_Gazi
{
    0 başvuru
    class Program
    {
        0 başvuru
        static void Main(string[] args)
        {

            object obj;
            obj = 5;
            Console.WriteLine(obj.GetType());

            obj = "B";
            Console.WriteLine(obj.GetType());

            obj = 5.25f;
            Console.WriteLine(obj.GetType());

            obj = false;
            Console.WriteLine(obj.GetType());

            obj = 12.75M;
            Console.WriteLine(obj.GetType());


            Console.ReadKey();
        }
    }
```

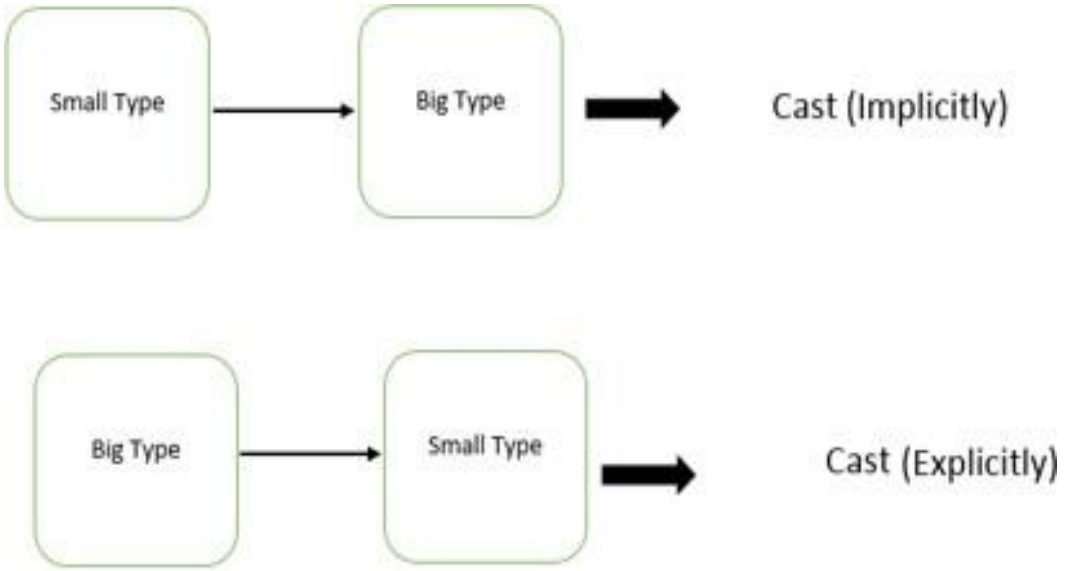
```
C:\Users\AKIN\source\repos\IE_Ga
System.Int32
System.String
System.Single
System.Boolean
System.Decimal
```

Type Casting (Conversion)

- Type conversion happens when we assign the value of one data type to another.
- If the data types are compatible, then C# does Automatic Type Conversion.
- If they are not comparable, then they need to be converted explicitly which is known as Explicit Type conversion.
- For example, assigning an int value to a long variable.

Type Casting (Conversion)





Implicit Type Casting / Automatic Type

It happens when:

- The two data types are compatible.
- When we assign value of a smaller data type to a bigger data type.

Convert from Data Type	Convert to Data Type
byte	short, int, long, float, double
short	int, long, float, double
int	long, float, double
long	float, double
float	double

Implicit Type Casting / Automatic Type

Try this ➔

Program.cs

IE_Gazi | IE_Gazi.Program

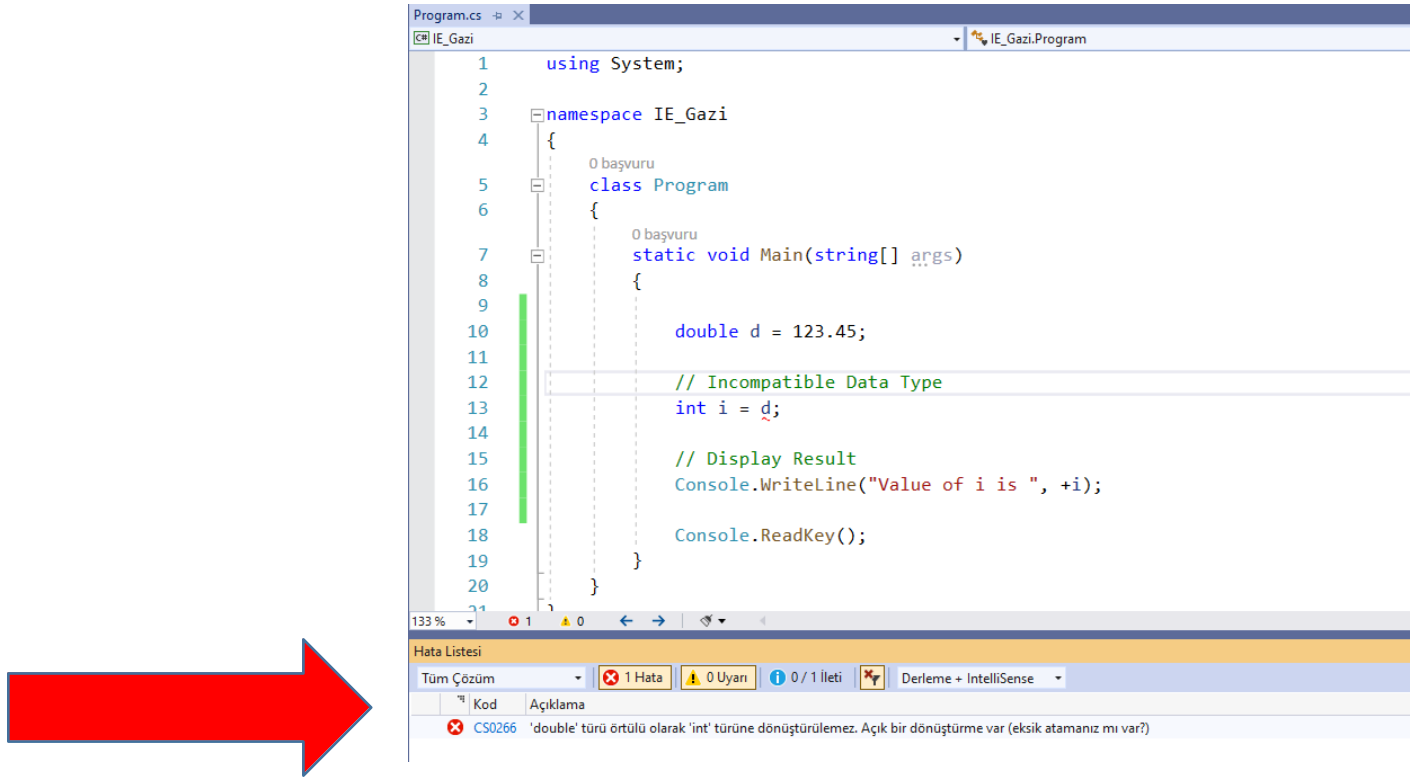
```
using System;

namespace IE_Gazi
{
    0 başvuru
    class Program
    {
        0 başvuru
        static void Main(string[] args)
        {

            int i = 25;

            // automatic type conversion
            long l = i;

            // automatic type conversion
            float f = l;

            // Display Result
            Console.WriteLine("Int value " + i);
            Console.WriteLine("Long value " + l);
            Console.WriteLine("Float value " + f);


            Console.ReadKey();
        }
    }
}
```

C:\Users\AKIN\source\repos\IE_Gazi\IE_Ga

```
Int value 25
Long value 25
Float value 25
```

Explicit Type Casting

There may be compilation error when types not compatible with each other. For example, assigning double value to int data type:



Explicit Type Casting

- So, if we want to assign a value of larger data type to a smaller data type, we perform an explicit type casting.
- This is useful for incompatible data types where automatic conversion cannot be done.
- Here, the target type specifies the desired type to which the specified value will be converted.
- Sometimes, it may result a lossy conversion.

Program.cs

IE_Gazi | IE_Gazi.Program

```
using System;

namespace IE_Gazi
{
    0 başvuru
    class Program
    {
        0 başvuru
        static void Main(string[] args)
        {

            double d = 123.45;

            // Explicit Type Casting
            int i = (int)d;

            // Display Result
            Console.WriteLine("Value of i is " + i);

            Console.ReadKey();
        }
    }
}
```

C:\Users\AKIN\source\repos\IE_Gazi\IE_Gaz

```
Value of i is 123
```

Object Type and ToString() Method

- All data types, including basic data types, derive from a reference type called an object.
- Since derivation is done by inheritance, existing traits are always preserved.
- In C#, everything is derived from the object reference type.
- There is always a class structure at its core.
- For example, the ToString() method of the Object Class is used in all the basic data reference types.

ToString() Method

- .ToString() method can be used on all base types or reference types. The aim here is the conversion to a string.
- string str = 123.45f.ToString()

23.ToString();

–12.34F.ToString();

Try these two on an online compiler

ToString() Method

- Try this ➔

Program.cs

IE_Gazi

```
using System;

namespace IE_Gazi
{
    0 başvuru
    class Program
    {
        0 başvuru
        static void Main(string[] args)
        {

            int a = 5;
            int b = 7;
            string a1 = a.ToString();
            string b1 = b.ToString();
            Console.WriteLine(a + b);
            Console.WriteLine(a1 + b1);

            Console.ReadKey();
        }
    }
}
```

Boxing – Unboxing

- Boxing and unboxing is an important concept in C#.
- We mentioned that C# Type System contains two main data types:
  Value Types (int, char, etc), Reference Types (object)
- Basically, it converts a Value Type to a Reference Type, and vice versa.
- Boxing and Unboxing enables a unified view of the type system in which a value of any type can be treated as an object.

Boxing	Unboxing
It convert value type into an object type.	It convert an object type into value type.
Boxing is an implicit conversion process.	Unboxing is the explicit conversion process.
Here, the value stored on the stack copied to the object stored on the heap memory.	Here, the object stored on the heap memory cop value stored on the stack .

Boxing - Unboxing

Boxing ➔

Program.cs

IE_Gazi | IE_Gazi.Program

```
4   {
    0 başvuru
5   class Program
6   {
    0 başvuru
7       static void Main(string[] args)
8       {
9
10          int val = 2022;
11
12          // Boxing
13          object o = val;
14
15          // Change the value of val
16          val = 1990;
17
18          Console.WriteLine("Value type of val is {0}", val);
19          Console.WriteLine("Object type of val is {0}", o);
20
21          Console.ReadKey();
22      }
23  }
24 }
```

C:\Users\AKIN\source\repos\IE_Gazi\IE_Gazi\bin\Debug\netcoreapp...

```
Value type of val is 1990
Object type of val is 2022
```

Boxing – Unboxing

- Unboxing ➔

Program.cs

IE_Gazi | IE_Gazi.Program

```
4   {
        0 başvuru
5       class Program
6       {
            0 başvuru
7           static void Main(string[] args)
8           {
9
10              int val = 2022;
11
12              // Boxing
13              object o = val;
14
15              // Unboxing
16              int x = (int)o;
17
18              Console.WriteLine("Value of o is {0}", o);
19              Console.WriteLine("Value of x is {0}", x);
20
21              Console.ReadKey();
22          }
23      }
24  }
```

C:\Users\AKIN\source\repos\IE_Gazi\IE_Gazi\bin\Debug\netcoreapp...

```
Value of o is 2022
Value of x is 2022
```

THE END

CHECK OUT COURSE WEBSITE: HTTPS://GAZI-END-MUH.GITHUB.IO

GOT ANY QUESTIONS LATER?

SEND ME AN EMAIL: DR.ERCAN.EZIN@GMAIL.COM